MOTOR HPR

Hoja de ruta técnica
Fase 5 y evolución posterior

Directrices de implementación del Motor de Pensamiento Verificable y plan de trabajo para la siguiente sesión.

Enfoque: arquitectura modular · discernimiento informacional · evaluación reproducible · seguridad por diseño

Propósito del documento

Establecer una guía de implementación para evolucionar el Motor HPR desde un pipeline funcional hacia un sistema que evalúe evidencia, contraste afirmaciones, represente incertidumbre y justifique sus decisiones. El documento define arquitectura, responsabilidades, etapas, criterios de aceptación y fases posteriores.

Principio rector: el motor no debe optimizarse para sonar convincente, sino para producir conclusiones proporcionales a la evidencia, reconocer sus límites y mantener los controles de seguridad.

Estado del documento: propuesta de ingeniería para la siguiente sesión. Los nombres de archivos y módulos son orientativos y deben contrastarse con el árbol real del repositorio antes de crear o modificar componentes.

1. Resumen ejecutivo

La siguiente iteración debe construir un primer núcleo verificable de razonamiento, sin intentar replicar el sistema propietario de ChatGPT ni reemplazar los componentes actuales de Nexus. La meta es diseñar una arquitectura propia, modular y evaluable que permita distinguir entre datos, afirmaciones, inferencias y conclusiones.

La Fase 5 se centra en los contratos de datos, la evaluación de afirmaciones, la política de decisión y la integración controlada. Las fases siguientes amplían progresivamente la comparación de hipótesis, la planificación adaptativa, la calibración y la memoria epistemológica.

Contexto de partida

- Tomar como referencia el último estado sincronizado del proyecto y el commit informado db3393c; verificar su existencia y contenido antes de actuar.
- Reutilizar el selector de triggers, query_builder.py, NexusRouter y MotorRazonamientoSuperior cuando sus contratos actuales lo permitan.
- Preservar Nexus Root y Sovereign Gate como controles de seguridad independientes y obligatorios.
- No declarar mejoras de precisión, razonamiento o confiabilidad sin resultados reproducibles.

2. Principios de arquitectura

- Modularidad: separar planificación, recuperación de evidencia, evaluación, verificación, decisión y presentación.
- Contratos explícitos: usar modelos de datos validados y evitar dependencias circulares o estado global implícito.
- Trazabilidad: cada conclusión importante debe poder vincularse a evidencia identificable y a una decisión documentada.
- Seguridad por diseño: los datos recuperados son contenido no confiable; no pueden alterar las instrucciones de control ni las políticas del sistema.
- Incertidumbre honesta: distinguir falta de evidencia, contradicción y evidencia contraria; no convertir ausencia de datos en prueba de falsedad.
- Coste acotado: limitar iteraciones, tiempo, consultas externas y consumo de recursos.
- Validación empírica: comparar el comportamiento nuevo con una línea base usando conjuntos de pruebas fijos.

3. Arquitectura lógica propuesta

Capa	Responsabilidad	Salida principal
Entrada y contexto	Interpretar intención, restricciones y ambigüedades.	Solicitud normalizada
Planificador	Descomponer objetivos y definir comprobaciones necesarias.	Plan acotado
Motor de evidencia	Recuperar, identificar y organizar fuentes y fragmentos.	Registro de evidencias
Discernimiento	Separar hechos, inferencias, hipótesis y contradicciones.	Evaluaciones justificadas
Verificador	Examinar el respaldo de las afirmaciones centrales.	Estado de verificación
Política de decisión	Elegir responder, investigar, aclarar o abstenerse.	Decisión y motivo
Presentación y auditoría	Comunicar conclusión, límites y trazas de decisión.	Respuesta fundamentada

Regla transversal: Nexus Root y Sovereign Gate conservan autoridad sobre permisos, integridad y ejecución. El discernimiento evalúa qué está justificado por la evidencia; no sustituye las políticas de seguridad.

4. Estructura de módulos sugerida

La estructura siguiente es orientativa. Antes de crear archivos, inspeccionar el repositorio y adaptar los nombres a los módulos existentes.

- reasoning/reasoning_engine.py: orquestación del ciclo completo.
- reasoning/planner.py: objetivos, subproblemas y presupuesto de comprobaciones.
- reasoning/evidence_manager.py: procedencia, pertinencia, duplicados y contradicciones.
- reasoning/hypothesis_evaluator.py: comparación de explicaciones alternativas.
- reasoning/claim_verifier.py: comprobación de afirmaciones centrales frente a evidencias.
- reasoning/uncertainty.py: lagunas, ambigüedad y calibración de confianza.
- reasoning/decision_policy.py: respuesta, investigación, aclaración o abstención.
- observability/reasoning_trace.py: eventos auditables sin registrar secretos ni datos sensibles innecesarios.
- models/: contratos validados para evidencias, afirmaciones y resultados.
- tests/: pruebas unitarias, de integración, de regresión y evaluación comparativa.

5. Fase 5 — Motor de Pensamiento Verificable

Objetivo

Implementar una primera capa de evaluación crítica que relacione afirmaciones con evidencias, identifique contradicciones, gestione información insuficiente y tome decisiones explícitas sobre la siguiente acción.

5.1 Contratos de datos

- Definir un modelo de evidencia con identificador estable, procedencia, contenido relevante, fecha cuando esté disponible y metadatos de recuperación.
- Definir un modelo de evaluación de afirmaciones con estado: supported, contradicted, unverified o insufficient_evidence.
- Definir un resultado estructurado con conclusión opcional, evaluaciones, decisión, limitaciones y referencias a evidencia.
- Validar los modelos con la versión real de Python/Pydantic del proyecto.
- No permitir que puntuaciones de relevancia se interpreten automáticamente como probabilidades de verdad.

5.2 Evaluador mínimo

- Separar la evaluación de evidencia de la generación de lenguaje natural.
- Implementar primero reglas explícitas y casos deterministas antes de añadir evaluación mediante un modelo generativo.
- Distinguir evidencia favorable, evidencia contradictoria y falta de evidencia.
- Registrar qué evidencia respalda cada afirmación y qué limitaciones permanecen.
- Evitar que una fuente única, repetida o de baja calidad sea tratada como confirmación independiente.

5.3 Política de decisión

- answer: responder cuando el nivel de evidencia satisface los criterios establecidos para el tipo de consulta.
- investigate: buscar evidencia adicional cuando la incertidumbre pueda reducirse dentro del presupuesto disponible.
- clarify: pedir datos al usuario cuando la ambigüedad afecte sustancialmente al resultado.
- abstain: no emitir una conclusión firme cuando no exista fundamento suficiente o el riesgo supere los límites aceptados.

5.4 Límites y seguridad

- Definir un máximo de iteraciones, tiempo y consultas por solicitud.
- Tratar páginas, documentos y resultados recuperados como datos no confiables; nunca ejecutar sus instrucciones como políticas internas.
- Mantener los controles de Nexus Root y Sovereign Gate en todas las rutas, incluidos errores y abstenciones.
- Registrar decisiones y códigos de motivo sin volcar credenciales, secretos ni contenido sensible innecesario.
- Ante una excepción de un módulo, fallar de forma controlada y no convertir el error en una respuesta factual.

5.5 Integración

- Conectar el evaluador al pipeline mediante una interfaz estable, sin reescribir todo el router.
- Mantener compatibilidad con los triggers y el flujo actual de la interfaz Nexus.
- Agregar pruebas de integración y regresión antes de activar el comportamiento por defecto.
- Si es viable, usar un modo de observación que calcule evaluaciones sin cambiar la respuesta visible; comparar resultados antes de habilitar decisiones automáticas.

Entregable de la Fase 5

Un núcleo mínimo que recibe afirmaciones y evidencias, devuelve evaluaciones trazables, decide entre responder/investigar/aclarar/abstenerse y supera una batería de pruebas reproducibles sin degradar los controles existentes.

6. Hoja de ruta para la siguiente sesión

Etapa	Trabajo	Criterio de salida
1. Auditoría y línea base	Inspeccionar árbol, commit de referencia, contratos actuales y pruebas existentes.	Mapa de dependencias y resultados iniciales registrados.
2. Contratos y modelos	Crear/adaptar modelos de evidencia, afirmaciones y resultado.	Validación de modelos y pruebas unitarias.
3. Evaluador mínimo	Implementar estados de evidencia y vínculo afirmación-evidencia.	Casos respaldados, contradictorios y no verificados reproducibles.
4. Política de decisión	Implementar answer/investigate/clarify/abstain y límites.	Decisiones con motivos verificables y ciclos acotados.
5. Integración controlada	Conectar con el pipeline y mantener las políticas de seguridad.	Pruebas de integración, UI y seguridad superadas.
6. Auditoría y cierre	Documentar resultados, fallos, deuda técnica y cambios.	Commit revisado y hoja de ruta actualizada.

Orden recomendado: no iniciar etapas posteriores si los contratos, la evaluación mínima y las pruebas de seguridad no son estables.

7. Fases posteriores a la Fase 5

Fase 6 — Razonamiento comparativo y planificación adaptativa

- Comparar hipótesis alternativas mediante criterios explícitos y evidencia discriminante.
- Seleccionar la siguiente consulta o comprobación según la laguna más relevante.
- Definir presupuestos máximos de llamadas, tiempo y coste.
- Probar casos donde distintas hipótesis explican los mismos datos y documentar cuándo no es posible decidir.
- Salida: planificador acotado y evaluación comparativa medible, sin ciclos autónomos ilimitados.

Fase 7 — Calibración, memoria epistemológica y evaluación longitudinal

- Construir un conjunto de referencia con respuestas evaluadas y criterios de puntuación explícitos.
- Medir fidelidad a la evidencia, detección de contradicciones, tasa de abstención apropiada y calibración de confianza.
- Guardar hechos verificados, procedencia, vigencia, hipótesis provisionales y errores conocidos en categorías separadas.
- Establecer reglas de expiración, actualización, corrección y retirada de memoria.
- Evaluar regresiones a lo largo de varias versiones y comparar con una línea base.
- Salida: informe de evaluación longitudinal y memoria con procedencia y mecanismos de revisión.

Fase 8 — Integración avanzada y validación integral

- Evaluar la interacción entre razonamiento, seguridad, interfaz, recuperación y memoria.
- Ejecutar pruebas de inyección de instrucciones, datos maliciosos, fallos parciales y fuentes contradictorias.
- Medir latencia, uso de recursos, coste por solicitud y degradación en escenarios de carga.
- Mantener revisión humana para decisiones sensibles y para cambios de política o conocimiento de alta importancia.
- Documentar limitaciones, amenazas conocidas y condiciones bajo las cuales el sistema no debe utilizarse.
- Salida: informe técnico de validación; no implica por sí solo que la hipótesis global de HPR esté demostrada.

8. Matriz de pruebas y métricas

Métrica	Qué evalúa	Cómo usarla
Fidelidad a la evidencia	Proporción de afirmaciones verificables con respaldo suficiente.	Revisar muestras con criterios consistentes; separar afirmaciones centrales y secundarias.
Detección de contradicciones	Capacidad de detectar conflictos presentes en el conjunto de referencia.	Medir precisión y cobertura frente a casos etiquetados.
Calibración	Correspondencia entre confianza declarada y frecuencia de acierto.	Usar un conjunto etiquetado y métricas de calibración; no validar con impresiones subjetivas.
Calidad de abstención	Si se abstiene cuando corresponde y responde cuando existe evidencia suficiente.	Medir falsos rechazos y conclusiones injustificadas conjuntamente.
Trazabilidad	Posibilidad de vincular conclusiones a fuentes y decisiones.	Auditar referencias, identificadores y motivos registrados.
Regresión de seguridad	Preservación de controles y límites de ejecución.	Ejecutar pruebas de regresión en cada integración.
Latencia y recursos	Coste temporal y computacional de la nueva capa.	Comparar con línea base y fijar presupuestos explícitos.

9. Casos mínimos de prueba

- Consulta respaldada: fuentes pertinentes y consistentes; el motor produce una conclusión vinculada a evidencia.
- Fuentes contradictorias: el motor identifica el conflicto y evita ocultarlo en una síntesis.
- Información insuficiente: el motor limita la conclusión o solicita información adicional.
- Hipótesis alternativas: compara explicaciones y no presenta una posibilidad como hecho.
- Contenido malicioso: las instrucciones dentro de documentos o páginas no cambian las políticas del agente.
- Fallo parcial: un conector o evaluador falla; el sistema comunica la limitación sin inventar resultados.
- Compatibilidad: triggers, selector, rutas multilingües e interfaz existente mantienen el comportamiento esperado.

10. Lista de verificación para la sesión

- ☐ Confirmar rama y estado del repositorio; revisar el commit de referencia antes de modificar archivos.
- ☐ Ejecutar la suite existente y registrar los resultados de línea base.
- ☐ Mapear los contratos actuales de NexusRouter, MotorRazonamientoSuperior y query_builder.py.
- ☐ Decidir los modelos de evidencia y evaluación antes de implementar la lógica.
- ☐ Implementar el evaluador mínimo con pruebas unitarias.
- ☐ Añadir estados de decisión y límites de iteración.
- ☐ Probar contradicciones, ausencia de evidencia, errores y contenido no confiable.
- ☐ Integrar detrás de un punto de entrada controlado; preferir modo de observación si reduce riesgo.
- ☐ Ejecutar pruebas de UI y seguridad sin saltarse controles.
- ☐ Documentar limitaciones y deuda técnica; solo después crear y sincronizar el commit.

11. Condiciones de cierre

- La Fase 5 se considera completada cuando sus criterios de aceptación están cubiertos por pruebas, no solo por una validación manual aislada.
- No se afirma que HPR tenga razonamiento humano ni que replique el motor interno de ChatGPT; se habla de capacidades funcionales concretas y evaluables.
- Toda mejora reportada debe estar respaldada por resultados comparables, condiciones de prueba y limitaciones explícitas.
- Los cambios se integran de forma incremental, con rollback posible y sin debilitar las fronteras de seguridad.

12. Cierre conceptual

El Motor de Pensamiento Verificable debe ayudar a HPR a pasar de la generación de respuestas a la evaluación disciplinada de conclusiones. Su valor no estará en la extensión de sus respuestas ni en la apariencia de certeza, sino en la capacidad de distinguir evidencia de inferencia, detectar límites, conservar trazabilidad y tomar decisiones proporcionales a lo que realmente puede sostener.

La colaboración aumentada constituye el principio de diseño: la IA amplía la capacidad humana de explorar y contrastar información, mientras que la supervisión humana, la evidencia y los controles verificables siguen siendo esenciales.

Documento de planificación técnica. Los módulos, métricas y etapas aquí descritos son propuestas que deben adaptarse al estado real del repositorio y validarse durante la implementación.